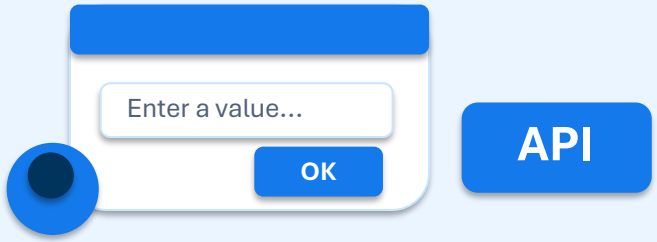
Input, Processing, and Output

1 Input

- Data entered by the user or received from another source
- Examples: prompt(), form fields, API data
- Raw information before any action is taken



API

2 Processing

- The program works with the input data
- Includes calculations, decisions, and transformations
- Happens through JavaScript statements and logic

```
const sum = num1 + num2;
if (sum > 10) {
  console.log("Large number");
}
```

3 Output

- The result produced after processing
- Examples: console.log(), alerts, page updates
- Information shown to the user or system

```
> console.log(sum);
5
```

The sum is 5!
OK

Result: 5

Example Flow

Input: user enters 2 numbers → Processing: add the numbers → Output: display the sum

Variables and Constants — let & const

1 Variables with let

- Used for values that may change
- Can be reassigned later in the program
- Block scoped

```
let score = 10;
score = 15;
```

10 → 15

Value can change (reassigned)

2 Constants with const

- Used for values that should stay the same
- Must be assigned when declared
- Block scoped

```
const pi = 3.14;
// cannot be reassigned
```

3.14

Binding cannot be reassigned

3 When to Use Each

- Use const by default
- Use let only when reassignment is needed
- Avoid var in modern JavaScript

const ✓

let

Best Practice

Prefer const for stable values and let for changing values.

Naming Rules and Conventions

1 Rules

- Names can use letters, digits, underscore (_), and dollar sign ($)
- Names cannot start with a digit
- Reserved keywords cannot be used as names

```
let studentName;
let _count;
let $price;
```

2 Conventions

- Use camelCase for variables and functions
- Choose clear and meaningful names
- Keep names consistent across the program

```
let studentName = "John";
let totalPrice = 59.99;
let isLoggedIn = true;
```

3 Good vs Bad Examples

Good	Bad
✓ studentName	✗ 1name
✓ totalPrice	✗ my-variable
✓ isLoggedIn	✗ x

Clear and meaningful

Remember

Readable names make code easier to understand and maintain.

JavaScript Data Types

Data types describe the kind of value stored in a variable.

String

Text data, written inside quotes.

```
"Hello"
```

Number

Numeric data (whole or decimal).

```
42
```

Boolean

Logical value: true or false.

```
true
```

Undefined

A variable that has no value yet.

```
undefined
```

Null

Intentional absence of any value.

```
null
```

BigInt

Integer numbers beyond safe Number limits.

```
9007199254740991n
```

Symbol

A unique, immutable value.

```
Symbol("id")
```

Object

A collection of key-value pairs.

```
{ name: "Ali" }
```

Why It Matters

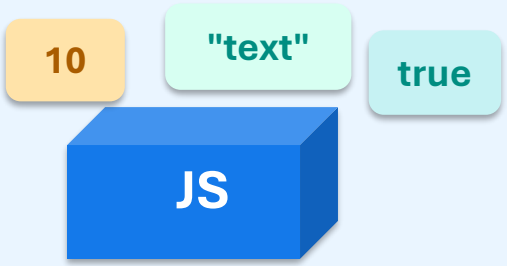
Understanding data types helps you store, process, and check values correctly.

Arrays and functions are special kinds of objects.

Dynamic Typing and typeof

1 Dynamic Typing

- A variable can hold different types at different times
- The type depends on the current assigned value
- No need to declare a fixed data type in advance



2 Example

```
let value = 10;
value = "ten";
value = true;
```

The same variable changes from number to string to boolean.

3 typeof Operator

```
typeof "Hello"  → "string"
typeof 42       → "number"
typeof true     → "boolean"
typeof undefined → "undefined"
```

Note: typeof null returns 'object' (historical quirk).

Key Takeaway

Use typeof to inspect the type of a value at runtime.

Assignment Operators

1 Basic Assignment

- = assigns a value to a variable

```
// Assign the value 10 to x
let x = 10;
```

10 → x

2 Shorthand Assignment

+=	add and assign
-=	subtract and assign
*=	multiply and assign
/=	divide and assign
%=	remainder and assign

3 Examples

```
let x = 10;

x += 5; // x = x + 5

x *= 2; // x = x * 2

x -= 3; // x = x - 3
```

+=

Why Use Them?

Assignment operators make code shorter and easier to read.

Arithmetic Operators

1 Main Operators

+	Addition
−	Subtraction
*	Multiplication
/	Division
%	Remainder





2 Increment and Decrement

++	Increase by 1
--	Decrease by 1

Example:

```
let count = 0;
count++; // increase by 1
count--; // decrease by 1
```

3 Examples

10 + 5	=	15
10 – 5	=	5
10 * 5	=	50
10 / 5	=	2
10 % 3	=	1

Key Point

Arithmetic operators are used for calculations in JavaScript.